ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 15

Bases de datos con PDO

- La despensa: qué es una base de datos relacional
- Nuestra base de datos: academia_alpe
- Repaso de SQL: las cuatro consultas
- Conectar con PDO
- Leer datos: query, fetch y fetchAll
- Consultas preparadas: el escudo contra la inyección SQL
- Escribir datos: INSERT, UPDATE y DELETE
- Transacciones: todo o nada
- Botiquín: los errores de base de datos más típicos



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

¿Os acordáis de la analogía del restaurante? El navegador es el cliente, PHP es el cocinero… y la base de datos es la **despensa**. Hasta ahora guardábamos los datos en ficheros JSON, que está bien para un libro de visitas, pero no para una academia con cientos de alumnos que se matriculan a la vez.

En la clase 6 ya conectasteis con MySQL. Hoy lo hacemos **bien**: con una conexión que no se rompe, con consultas preparadas que cierran la puerta a la inyección SQL y con **transacciones**, que garantizan que una operación se hace entera o no se hace.

> 📌 Antes de esta clase conviene haber visto los bloques S01 a S06 del itinerario de SQL. Aquí solo repasamos lo imprescindible.

•

👌 Cómo vamos a trabajar



1

📘 La despensa: qué es una base de datos relacional



2

🔍 Nuestra base de datos: academia_alpe



3

🤔 Repaso de SQL: las cuatro consultas



4

👍 Conectar con PDO



5

🚀 Leer datos: query, fetch y fetchAll



6

🐞 Consultas preparadas: el escudo contra la inyección SQL



7

👌 Escribir datos: INSERT, UPDATE y DELETE



8

⚠️ Transacciones: todo o nada



9

🐞 Botiquín: los errores de base de datos más típicos



10

🚀 Ejercicios



11

👋 Lo que tenéis que llevaros de hoy





CLASE 15

👌 Cómo vamos a trabajar

1. En Laragon, pulsad **Iniciar todo** (Apache **y** MySQL tienen que estar en verde).
2. Pulsad el botón **Base de datos**: se abre HeidiSQL. Conectad con el usuario `root` y la contraseña vacía.
3. Abrid la pestaña **Consulta**, pegad el contenido de `academia_alpe.sql` y pulsad **F9**. Se crea la base de datos `academia_alpe` con datos de prueba.
4. Trabajad en `php-paso-a-paso/clase14/`. Copiad también la carpeta `includes/`, con la conexión y las funciones de ayuda.



💡 El script empieza borrando la base de datos si ya existe. Así, si un día la dejáis hecha un desastre en una prueba, lo volvéis a ejecutar y queda como nueva.



CLASE 15 · PUNTO 1

📘 La despensa: qué es una base de datos relacional

Imaginad una hoja de cálculo enorme, preparada para que miles de personas lean y escriban a la vez sin estropear nada. Eso es, a grandes rasgos, una **base de datos relacional** como MySQL o MariaDB. La información se organiza en **tablas** [1]:

Concepto	Qué es	En nuestra academia
Tabla	Un tipo de cosa que guardamos	`alumnos`, `cursos`
Fila (registro)	Un elemento concreto	Marta García
Columna (campo)	Un dato, siempre del mismo tipo	`nombre`, `email`, `nota`
Clave primaria (PK)	El «DNI» de cada fila: un número único, normalmente `AUTO_INCREMENT`	`id`
Clave foránea (FK)	Una columna que apunta a la clave primaria de otra tabla	`curso_id` en `alumnos`

¿Por qué separar alumnos y cursos en dos tablas? Porque si guardarais el título y el precio del curso en cada alumno, al cambiar el precio tendríais que tocar cien filas. Con la clave foránea, el curso está **en un**

solo sitio y cada alumno solo guarda su número. Es el mismo principio DRY de las funciones, aplicado a los datos.



💡 **Dos convenciones que vais a ver:** en SQL, los nombres de tablas y columnas van en minúsculas con guion bajo (`curso_id` , `fecha_alta`), que es lo habitual en MySQL y en Laravel. En PHP seguimos con *camelCase*: `$cursoId` . Y el juego de caracteres siempre es `utf8mb4` : el antiguo `utf8` de MySQL no admite emojis ni algunos caracteres especiales [2].



CLASE 15 · PUNTO 2

🔍 Nuestra base de datos: academia_alpe

Esquema

```
cursos                          alumnos
┌──────────────────┐            ┌──────────────────┐
│ id          (PK) │◄───────────│ id          (PK) │
│ titulo           │            │ nombre           │
│ horas            │            │ email       (único)
│ precio           │            │ ciudad           │
│ plazas   (libres)│            │ nota        (o NULL)
└──────────────────┘            │ curso_id    (FK) │
                                │ fecha_alta       │
                                └──────────────────┘
```

Un curso tiene muchos alumnos; cada alumno está en un curso. Fijaos en tres detalles del script, porque los usaremos hoy:

- `email` es `UNIQUE` : MySQL no deja guardar dos alumnos con el mismo email.
- `curso_id` es una **clave foránea**: no podéis matricular a nadie en un curso que no existe.
- `plazas` son las plazas **libres**. El curso de bases de datos empieza completo, con 0.

📄 Script: `academia_alpe.sql`



CLASE 15 · PUNTO 3

🤔 Repaso de SQL: las cuatro consultas

SQL es el idioma de las bases de datos. Cuatro sentencias hacen casi todo el trabajo, y sus iniciales en inglés forman la palabra **CRUD** [3]:

Operación	SQL	Ejemplo
Crear	`INSERT`	`INSERT INTO cursos (titulo, horas, precio) VALUES ('Git', 20, 60.00);`
Leer (*Read*)	`SELECT`	`SELECT nombre, email FROM alumnos WHERE ciudad = 'Santander' ORDER BY nombre;`
Actualizar (*Update*)	`UPDATE`	`UPDATE alumnos SET nota = 9.5 WHERE id = 1;`
Borrar (*Delete*)	`DELETE`	`DELETE FROM alumnos WHERE id = 1;`

Y para unir tablas, `INNER JOIN`: «dame cada alumno **con** el título de su curso».

consulta.sql — php-paso-a-paso/clase15 — Visual Studio Code

consulta.sql

```sql
SELECT a.nombre, a.email, c.titulo AS curso
FROM alumnos a
INNER JOIN cursos c ON a.curso_id = c.id
ORDER BY a.nombre;
```

La `a` y la `c` son **alias**: apodos cortos para no escribir el nombre de la tabla entero cada vez. `AS curso` cambia el nombre de la columna en el resultado.



😱 **La pesadilla del `UPDATE` y el `DELETE` sin `WHERE`.** `DELETE FROM alumnos;` no borra un alumno: **los borra todos**. `UPDATE alumnos SET nota = 0;` suspende a toda la academia. Antes de ejecutar un `UPDATE` o un `DELETE`, comprobad dos veces el `WHERE`.



CLASE 15 · PUNTO 4

👍 Conectar con PDO

PDO (*PHP Data Objects*) es la forma estándar de conectar PHP con una base de datos [4]. Funciona con MySQL, MariaDB, PostgreSQL, SQLite... y es la que usa Laravel por dentro.



🚫 Si veis por internet `mysql_connect()` o `mysql_query()`, huid: esas funciones desaparecieron en PHP 7 por inseguras.

Para conectar hacen falta tres cosas: el **DSN** (una cadena que dice qué motor, qué servidor, qué base de datos y qué juego de caracteres), el usuario y la contraseña [5]. Como lo vamos a necesitar en todas las páginas, lo metemos en una función, en su propio archivo:

conexion.php — php-paso-a-paso/clase15 — Visual Studio Code

conexion.php

```php
<?php
declare(strict_types=1);

const DB_HOST = "localhost";
const DB_NOMBRE = "academia_alpe";
const DB_USUARIO = "root";
const DB_CLAVE = "";

function conectar(): PDO
{
    $dsn = "mysql:host=" . DB_HOST . ";dbname=" . DB_NOMBRE . ";charset=utf8mb4";
    $opciones = [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES => false,
    ];

    try {
        return new PDO($dsn, DB_USUARIO, DB_CLAVE, $opciones);
    } catch (PDOException $e) {
        http_response_code(500);
        exit("Error al conectar con la base de datos: " . $e->getMessage());
    }
}
```

📄 Archivo: `includes/conexion.php`

Las tres opciones que ponemos siempre

Opción	Qué hace	Por qué
`ERRMODE_EXCEPTION`	Convierte cualquier error de SQL en una **excepción**	Si no, PDO falla en silencio y no os enteráis
`FETCH_ASSOC`	Cada fila llega como array asociativo: `$fila["email"]`	Es lo que ya sabéis manejar desde la clase 11
`EMULATE_PREPARES => false`	Las consultas preparadas las hace MySQL de verdad	Más seguridad y los números llegan como números

Y en cada página, al principio:

01-probar-conexion.php — php-paso-a-paso/clase15 — Visual Studio Code

01-probar-conexion.php

```php
<?php
declare(strict_types=1);

require_once __DIR__ . "/includes/conexion.php";

$pdo = conectar();
```

📄 Ejemplo: `01-probar-conexion.php`



⚠️ En Laragon, `root` no tiene contraseña. En un servidor de verdad **siempre** la tiene, y además no se usa `root` para la web: se crea un usuario con los permisos justos.



CLASE 15 · PUNTO 5

🚀 Leer datos: query, fetch y fetchAll

Sin datos de fuera: query()

Si la consulta **no lleva ningún dato** que venga del usuario, podéis usar `query()` directamente:

02-listar-cursos.php — php-paso-a-paso/clase15 — Visual Studio Code

02-listar-cursos.php

```php
<?php
$cursos = $pdo->query("SELECT id, titulo, horas, precio, plazas FROM cursos ORDER BY
titulo")->fetchAll();

foreach ($cursos as $curso) {
    echo $curso["titulo"];
}
```

`fetchAll()` devuelve **todas** las filas en un array de arrays: exactamente la «tabla de datos» que manejasteis en la clase 11 [6].

📄 Ejemplos: `02-listar-cursos.php` y `03-alumnos-con-curso.php` (con `INNER JOIN`)

Con datos de fuera: prepare(), execute() y fetch()

En cuanto la consulta lleve un dato del usuario (de la URL, de un formulario...), se hace en **tres pasos**:

04-ficha-alumno.php — php-paso-a-paso/clase15 — Visual Studio Code

04-ficha-alumno.php

```php
<?php
$id = (int) ($_GET["id"] ?? 0);

$consulta = $pdo->prepare("SELECT nombre, email FROM alumnos WHERE id = :id");   // 1. preparar
$consulta->execute(["id" => $id]);                                               // 2. ejecutar
$alumno = $consulta->fetch();                                                    // 3. recoger

if ($alumno === false) {
    http_response_code(404);
    echo "Ese alumno no existe";
}
```

`:id` es un **marcador**: un hueco en la consulta que se rellena en `execute()` con el array de datos.

Método	Devuelve	Cuándo
`fetch()`	**Una** fila, o `false` si no hay ninguna	Buscar por `id`: una ficha
`fetchAll()`	**Todas** las filas (un array vacío si no hay)	Listados
`fetchColumn()`	El valor de la primera columna	Un `COUNT(*)` o un dato suelto

📄 Ejemplo: `04-ficha-alumno.php`. Probad con `?id=3` y con `?id=999`.



💡 **Un detalle de MySQL:** las columnas `DECIMAL` (precios, notas) llegan a PHP como **texto**, para no perder decimales. Por eso en los ejemplos veréis `euros((float) $curso["precio"])`. Y una nota vacía llega como `null`: de ahí el `$alumno["nota"] ?? "sin nota"`.



CLASE 15 · PUNTO 6

🐞 Consultas preparadas: el escudo contra la inyección SQL

En la clase 13 os presenté la inyección SQL. Hoy la vais a ver funcionar. Imaginad un buscador por email que **pega** el dato del usuario dentro del SQL:

05-inyeccion-sql.php — php-paso-a-paso/clase15 — Visual Studio Code

05-inyeccion-sql.php

```php
<?php
$sql = "SELECT nombre, email FROM alumnos WHERE email = '{$email}'";
```

Ahora alguien escribe esto en el campo del email:

Esquema

```
nadie@ejemplo.com' OR '1'='1
```

Y la consulta que llega a MySQL es:

consulta.sql — php-paso-a-paso/clase15 — Visual Studio Code

consulta.sql

```sql
SELECT nombre, email FROM alumnos WHERE email = 'nadie@ejemplo.com' OR '1'='1'
```

Como `'1'='1'` siempre es verdad, devuelve **la tabla entera** [7]. Con un poco más de maña, un atacante podría leer contraseñas o borrar datos.

Con una consulta preparada, el dato viaja **por separado** y MySQL lo trata como un simple texto, aunque lleve comillas:

05-inyeccion-sql.php — php-paso-a-paso/clase15 — Visual Studio Code

05-inyeccion-sql.php

```php
<?php
$consulta = $pdo->prepare("SELECT nombre, email FROM alumnos WHERE email = :email");
$consulta->execute(["email" => $email]);   // 0 resultados: no hay nadie con ese «email» tan raro
```

📄 Ejemplo: `05-inyeccion-sql.php`. Ejecuta las dos versiones y os enseña cuántas filas devuelve cada una.



🛡️ **La regla:** si en la consulta interviene **cualquier** dato que no hayáis escrito vosotros (formularios, URL, cookies, sesión…), `prepare()` y `execute()`. Sin excepciones. `query()` solo para consultas fijas.

Marcadores con nombre, siempre

PDO también admite marcadores con interrogación (`WHERE id = ?`), pero os pido que uséis **marcadores con nombre** (`:id`, `:email`). Con un formulario de diez campos y diez interrogaciones, equivocarse de orden es facilísimo; con nombres, el código se explica solo.



CLASE 15 · PUNTO 7

👌 Escribir datos: INSERT, UPDATE y DELETE

Para escribir, el proceso es el mismo: `prepare()` y `execute()`. Cambian las preguntas que podéis hacer después:

Método	Qué os dice	Ejemplo
`$pdo->lastInsertId()`	El `id` que MySQL acaba de dar al nuevo registro	Tras un `INSERT`
`$consulta->rowCount()`	Cuántas filas se han modificado o borrado	Tras un `UPDATE` o un `DELETE`

Alta de un alumno

06-alta-alumno.php — php-paso-a-paso/clase15 — Visual Studio Code

06-alta-alumno.php

```php
<?php
$consulta = $pdo->prepare(
    "INSERT INTO alumnos (nombre, email, curso_id) VALUES (:nombre, :email, :cursoId)"
);
$consulta->execute(["nombre" => $nombre, "email" => $email, "cursoId" => $cursoId]);
$nuevoId = (int) $pdo->lastInsertId();
```

En el ejemplo completo, el formulario sigue el orden de siempre (funciones, inicialización, entradas, procesamiento y salidas) y aplica *Post/Redirect/Get* tras guardar. Además, aprovecha un modo de lectura muy cómodo para los `<select>`: `fetchAll(PDO::FETCH_KEY_PAIR)` devuelve directamente un array `[id => título]`.

¿Y si el email ya existe? Como la columna es `UNIQUE`, MySQL se niega y PDO lanza una `PDOException` con el código **1062**. La capturamos para mostrar un mensaje amable:

06-alta-alumno.php — php-paso-a-paso/clase15 — Visual Studio Code

06-alta-alumno.php

```php
<?php
try {
    $consulta->execute([...]);
} catch (PDOException $e) {
    if (($e->errorInfo[1] ?? 0) === 1062) {
        $errores[] = "Ya hay un alumno con ese email.";
    } else {
        throw $e;   // cualquier otro error, que siga su camino
    }
}
```

📄 Ejemplo: `06-alta-alumno.php`

Cambiar notas y borrar alumnos

07-gestionar-notas.php — php-paso-a-paso/clase15 — Visual Studio Code

07-gestionar-notas.php

```php
<?php
$consulta = $pdo->prepare("UPDATE alumnos SET nota = :nota WHERE id = :id");
$consulta->execute(["nota" => $nota, "id" => $id]);

$consulta = $pdo->prepare("DELETE FROM alumnos WHERE id = :id");
$consulta->execute(["id" => $id]);
$borrado = $consulta->rowCount() === 1;
```

📄 Ejemplo: `07-gestionar-notas.php`



🚫 **Nunca se borra por GET.** Un enlace como `borrar.php?id=5` lo puede «pulsar» cualquiera: un buscador que recorre vuestra web, una vista previa de un chat o una imagen trampa en otra página. Borrar es una acción que **cambia datos**, así que va siempre en un formulario con `method="post"`. Y si podéis, con una confirmación antes.



CLASE 15 · PUNTO 8

⚠️ Transacciones: todo o nada

Pensad en una transferencia bancaria: hay que restar 100 € de una cuenta y sumarlos a otra. ¿Y si el servidor se cae justo entre las dos operaciones? Los 100 € desaparecerían por el camino. 😬

Una **transacción** agrupa varias consultas para que se hagan **todas o ninguna** [8]:

Método	Qué hace
`$pdo->beginTransaction()`	«A partir de aquí, nada es definitivo»
`$pdo->commit()`	«Todo ha ido bien: guárdalo»
`$pdo->rollBack()`	«Algo ha fallado: deshazlo todo, como si no hubiera pasado nada»

En nuestra academia, **matricular** son dos operaciones: gastar una plaza del curso y guardar al alumno. Si el email está repetido, el alumno no se guarda… pero la plaza ya se había restado. Con una transacción, el `rollBack()` devuelve también la plaza:

08-matricula-transaccion.php — php-paso-a-paso/clase15 — Visual Studio Code

08-matricula-transaccion.php

```php
<?php
function matricular(PDO $pdo, string $nombre, string $email, int $cursoId): void
{
    $pdo->beginTransaction();

    try {
        $consulta = $pdo->prepare("UPDATE cursos SET plazas = plazas - 1 WHERE id = :id AND
plazas > 0");
        $consulta->execute(["id" => $cursoId]);
        if ($consulta->rowCount() === 0) {
            throw new RuntimeException("No quedan plazas en ese curso.");
        }

        $consulta = $pdo->prepare(
            "INSERT INTO alumnos (nombre, email, curso_id) VALUES (:nombre, :email,
:cursoId)"
        );
        $consulta->execute(["nombre" => $nombre, "email" => $email, "cursoId" => $cursoId]);

        $pdo->commit();
    } catch (Throwable $e) {
        $pdo->rollBack();
        throw $e;
    }
}
```

Fijaos en el truco del `WHERE ... AND plazas > 0`: si no quedan plazas, el `UPDATE` no modifica ninguna fila, `rowCount()` devuelve 0 y lanzamos nuestra propia excepción.

📄 Ejemplo: `08-matricula-transaccion.php`

✅ Tres pruebas para comprobarlo

Haced esto	Debéis obtener
Matriculad a alguien nuevo en Laravel	«Matrícula completada» y una plaza menos
Matriculad otra vez con el **mismo email**	«Ya hay un alumno con ese email» y **las plazas no cambian**
Intentad matricular en «Bases de datos con MySQL»	«No quedan plazas»



💡 En el ejemplo, el `catch (PDOException $e)` va **antes** que el `catch (RuntimeException $e)`. ¿Por qué? Porque una `PDOException` es un tipo concreto de `RuntimeException`, y PHP usa el primer `catch` que encaja. Lo entenderéis del todo con la herencia, en la parte de programación orientada a objetos.



CLASE 15 · PUNTO 9

🐞 Botiquín: los errores de base de datos más típicos

Mensaje	Qué pasa	Solución
`could not find driver`	PHP no tiene activada la extensión `pdo_mysql`	Laragon: **Menú → PHP → Extensiones → pdo_mysql**
`Access denied for user 'root'`	Usuario o contraseña incorrectos	Revisad `DB_USUARIO` y `DB_CLAVE`
`Unknown database 'academia_alpe'`	La base de datos no existe	Ejecutad el script SQL en HeidiSQL
`Connection refused`	MySQL está parado	Pulsad **Iniciar todo** en Laragon
`Duplicate entry ... for key 'email'` (1062)	Valor repetido en una columna `UNIQUE`	Capturadlo y avisad al usuario
`a foreign key constraint fails` (1451 / 1452)	Un `curso_id` que no existe, o borrar un curso que tiene alumnos	Comprobad los datos relacionados
`Invalid parameter number`	Los marcadores del SQL no coinciden con las claves de `execute()`	Revisad que `:email` tenga su `"email" ⇒ ...`
`Unknown column 'nombre_alumno'`	Una columna mal escrita	Comparad con la tabla en HeidiSQL



CLASE 15 · PUNTO 10

🚀 Ejercicios

- ☐ **01 · SQL a mano.** En HeidiSQL: los alumnos de Santander ordenados por nombre; un curso nuevo con un alumno; un `INNER JOIN` con nombre, email y curso; e intentad matricular a alguien en el curso 999. ¿Qué dice MySQL? → `ej01-consultas.sql`
- ☐ **02 · Buscador.** Un formulario GET que busque alumnos cuyo nombre **contenga** el texto escrito. Pista: `LIKE :texto` y los `%` van en el dato, no en el SQL. → `ej02-buscador.php`
- ☐ **03 · Ficha de un curso.** `?id=1` muestra los datos del curso y la lista de sus alumnos. Si el curso no existe, error 404. → `ej03-ficha-curso.php`
- ☐ **04 · Estadísticas.** Para cada curso, cuántos alumnos tiene, su nota media y la mejor nota. Pista: `LEFT JOIN`, `COUNT`, `AVG`, `MAX` y `GROUP BY`. → `ej04-estadisticas.php`
- ☐ **05 · Cambio de curso.** Cambiad a un alumno de curso con una transacción: el curso nuevo pierde una plaza (si le quedan), el antiguo la recupera y el alumno cambia su `curso_id`. → `ej05-cambiar-curso.php`

- ☐ **06 · Paginación.** El listado de alumnos de 3 en 3 con enlaces a cada página (`?pagina=2`). Pistas: `COUNT(*)` con `fetchColumn()` , `LIMIT :cuantos OFFSET :desde` y `bindValue(..., PDO::PARAM_INT)` . → `ej06-paginacion.php`
- ☐ **07 · Alta de curso.** Un formulario que valide el título, las horas y el precio, inserte el curso con una consulta preparada y redirija con el id nuevo (`lastInsertId()`). → `ej07-alta-curso.php`
- ☐ **08 · Poner notas.** Un desplegable con los alumnos y un campo de nota: validad que está entre 0 y 10 y, si se deja vacío, guardad `NULL` . Usad `rowCount()` para saber si ha cambiado algo. → `ej08-poner-nota.php`
- ☐ **09 · Borrar con confirmación.** Un botón «Borrar» por alumno, que pida confirmación, borre por POST y avise de cuántas filas se han borrado. → `ej09-borrar-alumno.php`
- ☐ **10 · Filtros combinados.** Un desplegable de ciudades (sacadas con `SELECT DISTINCT`) y una casilla «solo aprobados». Montad el `WHERE` según los filtros elegidos, pero con los valores **siempre** como parámetros. → `ej10-filtros.php`
- ☐ **11 · Alumnos a Excel.** Descargad en CSV los alumnos de un curso (`?curso=1`), juntando lo de hoy con lo de la clase 12. → `ej11-exportar-csv.php`
- ☐ **12 · El podio.** Las tres mejores notas, con el nombre del curso y una medalla para cada puesto. → `ej12-ranking.php`



🏆 **Reto:** en el ejercicio 05, quitad la transacción (`beginTransaction` , `commit` y `rollBack`) y provocad un fallo a propósito justo antes del último `UPDATE` con un `throw new RuntimeException("¡Fallo!");` . Mirad las plazas en HeidiSQL. ¿A que ya no cuadran? Volved a poner la transacción y repetid la prueba.



CLASE 15 · PUNTO 11

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
Tablas relacionadas	Cada cosa en su tabla, unidas por claves foráneas
`utf8mb4`	El juego de caracteres de verdad, con tildes y emojis
Conexión	Una función `conectar()` con `ERRMODE_EXCEPTION`, `FETCH_ASSOC` y sin emulación
`query()`	Solo para consultas sin datos de fuera
`prepare()` + `execute()`	Para todo lo demás, con marcadores con nombre (`:id`)
`fetch()` / `fetchAll()`	Una fila (o `false`) / todas las filas
`lastInsertId()` / `rowCount()`	El id nuevo / cuántas filas han cambiado
`UPDATE` y `DELETE`	Siempre con `WHERE`; borrar, siempre por POST
Transacciones	`beginTransaction()`, `commit()` y, si algo falla, `rollBack()`

📚 Fuentes y documentación oficial

1. MySQL 8.0 Reference Manual: Tutorial
2. MySQL 8.0 Reference Manual: The utf8mb4 Character Set
3. MySQL 8.0 Reference Manual: Data Manipulation Statements
4. Manual de PHP: PDO (PHP Data Objects)
5. Manual de PHP: PDO_MYSQL DSN
6. Manual de PHP: PDOStatement::fetchAll
7. OWASP: SQL Injection
8. Manual de PHP: transacciones y auto-commit